



CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

Continuum AI – Knowledge Continuity Platform

Table of Contents

I. Record of Changes	3
II. Project Management Plan	4
1. Overview	4
1.1 Scope & Estimation	4
1.2 Project Objectives	6
1.3 Project Risks	7
2. Management Approach	8
2.1 Project Process	8
2.2 Quality Management	9
2.3 Training Plan	10
3. Project Deliverables	10
4. Responsibility Assignments	11
5. Project Communications	11
6. Configuration Management	11
6.1 Document Management	11
6.2 Source Code Management	12
6.3 Tools & Infrastructures	13

I. Record of Changes

Date	A*, M, D	In charge	Change Description
21/09/2026	A	[TBD]	Init version
21/09/2026	A	[TBD]	Add scope, objectives, risks, process, quality, deliverables and configuration management

*A - Added M - Modified D - Deleted

II. Project Management Plan

Highlighted [TBD] markers indicate information that is not available in the project documents (Document, DATN-BE, DATN-FE) and must be provided by the team.

1. Overview

1.1 Scope & Estimation

The scope follows the accepted Continuum AI MVP baseline (10-week vertical slice). The work breakdown structure below groups the work by feature, based on the functional requirements. Complexity and estimated effort (man-days) are not defined in the source documents.

#	WBS Item	Complexity	Est. Effort (man-days)
1	Feature 1: Identity, Access & Project Management		[TBD]
1.1	Authentication and session (JWT access token, rotating refresh token)	[TBD]	[TBD]
1.2	Three-role RBAC and scoped assignments (SME, Knowledge Owner, Successor)	[TBD]	[TBD]
1.3	Project, team and membership management; onboarding/offboarding states	[TBD]	[TBD]
1.4	project.create capability grant (issue, revoke, expire, audit)	[TBD]	[TBD]
1.5	Effective-permission evaluation and source/resource ACL	[TBD]	[TBD]
2	Feature 2: Knowledge Capture		[TBD]
2.1	Manual structured knowledge entry and required-knowledge templates	[TBD]	[TBD]
2.2	Daily / task notes (what, how, why, blockers, next steps)	[TBD]	[TBD]
2.3	Reminders for missing or overdue notes and required knowledge	[TBD]	[TBD]
3	Feature 3: Jira Cloud Integration		[TBD]
3.1	Connection and Jira-project mapping	[TBD]	[TBD]
3.2	Backfill of issues, comments and status	[TBD]	[TBD]
3.3	Idempotent webhook updates	[TBD]	[TBD]
3.4	Reconciliation and sync-failure handling	[TBD]	[TBD]
4	Feature 4: Source Upload & Ingestion		[TBD]
4.1	File upload (PDF, DOCX, Markdown, TXT, image) and Cloudflare R2 storage	[TBD]	[TBD]
4.2	Parsing / OCR / chunking with job retry and status	[TBD]	[TBD]
4.3	SAG indexing, source trace and search adapter	[TBD]	[TBD]
5	Feature 5: AI Extraction & Knowledge Lifecycle		[TBD]

#	WBS Item	Complexity	Est. Effort (man-days)
5.1	Proposed Knowledge extraction with evidence (structured schema)	[TBD]	[TBD]
5.2	Knowledge verification (approve, edit, reject, request clarification)	[TBD]	[TBD]
5.3	Versioning, supersede / deprecate and validity interval	[TBD]	[TBD]
6	Feature 6: Permission-aware Assistant		[TBD]
6.1	Permission-aware retrieval, cited answer and verification status/date	[TBD]	[TBD]
6.2	Insufficient-evidence response	[TBD]	[TBD]
6.3	Gap detection and follow-up	[TBD]	[TBD]
7	Feature 7: Handover		[TBD]
7.1	Member handover: scoped checklist, successor assignment, readiness confirmation	[TBD]	[TBD]
7.2	Successor onboarding: scoped handover package and cited assistant	[TBD]	[TBD]
8	Feature 8: Audit & Administration		[TBD]
8.1	Audit log (technical, security and project/team scope)	[TBD]	[TBD]
9	Feature 9: Evaluation & Benchmark		[TBD]
9.1	Labeled benchmark dataset and question set	[TBD]	[TBD]
9.2	Baselines (vector RAG, SAG, Continuum) and ablation	[TBD]	[TBD]
9.3	Retrieval, answer, security and performance metrics	[TBD]	[TBD]
10	Testing & Quality Assurance		[TBD]
10.1	Unit, integration, end-to-end, AI evaluation, security, performance and reliability testing	[TBD]	[TBD]
11	DevOps & Deployment		[TBD]
11.1	CI (GitHub Actions) and reproducible Docker Compose deployment	[TBD]	[TBD]
12	Documentation & Reports		[TBD]
12.1	Proposal / SRS / SDD / backlog and demo plan (based on the baseline specification)	[TBD]	[TBD]
12.2	Architecture, data model, API contract, test plan and benchmark dataset documents	[TBD]	[TBD]
	Total Estimated Effort (man-days)		[TBD]

1.2 Project Objectives

Continuum AI addresses knowledge inheritance when a leader or member leaves a software project, changes team or transfers responsibility. The core hypothesis is that a project team can reduce knowledge loss and successor time-to-information by continuously capturing, verifying, maintaining and transferring member knowledge through a structured workflow and an evidence-grounded RAG assistant.

Success criteria

- At least one end-to-end flow runs on real data: source document → SAG index → Proposed Knowledge → human verification → Verified Knowledge → AI answer with citation.
- A benchmark compares at least a vector-RAG baseline, SAG and the Continuum validation pipeline.
- Unauthorized evidence leakage is zero in the security test suite; restricted evidence is not sent to the LLM.
- When evidence is missing, outdated or unverified, the system returns an insufficient-evidence result instead of inventing an answer.
- Missing or overdue required knowledge creates a visible follow-up action.
- A member departure produces a prioritized handover plan based on real gaps; a successor receives a scoped handover package and can ask cited questions.
- The benchmark reports retrieval and answer quality against labeled expected outputs, and the handover evaluation measures time-to-information, answer correctness or unresolved gaps.
- Architecture, data model, API contract, test plan and benchmark dataset documents exist, and deployment is reproducible with Docker.

Measurable targets defined in the specification

Area	Target
Retrieval latency	p95 < 2 s for normal retrieval in the benchmark environment (LLM generation latency measured separately)
Normal AI response	2–8 s; complex multi-hop / graph queries may target 5–15 s
Unauthorized evidence leakage	0 in the security test suite
Citation	Every factual answer has evidence or an explicit insufficient-evidence result
Automatic verification	AI cannot set VERIFIED/ACTIVE or resolve a critical conflict on its own
Job reliability	Ingestion/extraction supports retry, status tracking, idempotency and failure reason

These are technical targets of the project, not results already achieved; final results must be measured on the team's benchmark.

Quality-by-stage targets (review / unit / integration / system coverage and defect limits), milestone timeliness and allocated effort per activity: **[TBD]** – not defined in the source documents.

1.3 Project Risks

Risks and mitigations are taken from the specification's risk register. The possibility rating is not defined in the source documents.

#	Risk Description	Impact	Possibility	Response Plans
1	Scope is too large	The core flow is not completed	[TBD]	- Lock P0 features; advanced onboarding/offboarding is a stretch goal - Demonstrate 4–5 flows instead of every module
2	SAG upstream changes	The integration breaks	[TBD]	- Pin the SAG commit/tag - Keep an adapter boundary and integration tests - Fork only if required
3	AI extraction is wrong	Knowledge quality is poor	[TBD]	- Structured schema and evidence requirement - Human review - Acceptance metrics
4	Hallucination	Loss of trust	[TBD]	- Evidence-only answer policy - Insufficient-evidence state - Citation tests
5	Permission leak	Serious security failure	[TBD]	- Pre-retrieval ACL and tenant/source scope - Negative security tests
6	Dataset has no ground truth	Effectiveness cannot be proven	[TBD]	- Create the golden dataset early - Label it manually - Compare with baselines
7	LLM cost or availability	Demo is unstable	[TBD]	- Provider abstraction - Cached evaluation - Local/mock fallback for non-AI paths
8	Contribution unclear because open source is reused	The committee rates the team's part lower	[TBD]	- SAG vs Continuum contribution table - Commit history, ADRs and benchmark - Domain features built by the team
9	Graph / entity duplicates	Retrieval noise	[TBD]	- Normalization and de-duplication - Namespace/type rules - Evaluate failure cases
10	Sensitive real data	Privacy / compliance risk	[TBD]	- Synthetic or anonymized capstone dataset - Retention and access policy

2. Management Approach

2.1 Project Process

The project is delivered as a 10-week plan in six phases (Figure 1). P0 is the vertical-slice foundation with real data; P1 features are accepted only when P0 is stable.

Phase	Week	Deliverables
P0 – Foundation	1	Scope/actor decisions, DB/API contracts, dataset design, Docker baseline and module ownership
P1 – Access & project	2–3	Auth, 3 roles, <code>project.create</code> grant, project/team membership, ACL and audit
P2 – Capture & integration	3–5	Manual note, Jira backfill/webhook/reconciliation, R2 upload, parse/OCR/job status
P3 – Knowledge lifecycle	5–7	SAG index/trace, Proposed Knowledge, human verification, immutable version/evidence
P4 – Chat & handover	7–8	Permission-aware chat with citation and insufficient evidence, gap, scoped handover checklist
P5 – Evaluation & hardening	9–10	Dataset benchmark, leak tests, failure analysis, bug fixes, reproducible demo and report

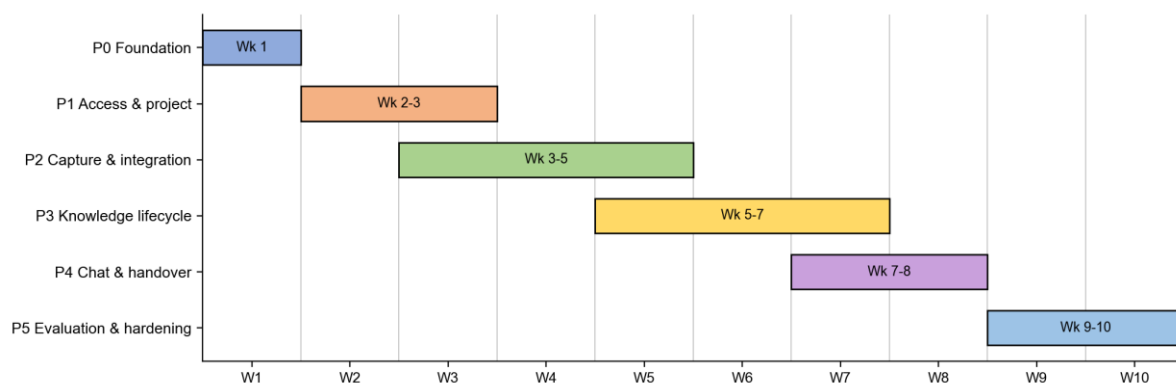


Figure 1. Delivery phases over the 10-week plan.

Every task follows the workflow defined in `AI_WORKFLOW.md`: receive the Jira task, read the rules and architecture, create a new branch from the latest `origin/main`, implement, test locally, commit, push, open a pull request into `main`, link it to Jira, get review and approval, and merge by an authorized teammate (Figure 2, section 6.2). API behavior is contract-first: request, response, error, scope, pagination and streaming semantics are approved in OpenAPI before implementation.

Development methodology (for example Scrum), iteration length and ceremonies: **[TBD]** – not defined in the source documents.

2.2 Quality Management

Testing strategy

Test level	Scope	Examples
Unit	Domain rules, validation, mapping, risk formula	Cannot verify without reviewer; validity overlap; permission scope builder
Integration	Core API ↔ MongoDB ↔ SAG ↔ LLM mock	Ingest mapping, search adapter, citation trace
E2E	User flows on UI/API	Upload → verify → ask; gap → follow-up → resolve
AI evaluation	Golden dataset and judge rubric / human review	Retrieval recall, faithfulness, extraction accuracy
Security	Auth/RBAC/ACL, prompt injection, data leakage	Finance source inaccessible to Sales; malicious document cannot override system policy
Performance	Load, retrieval, ingestion	Concurrent queries, large document batch
Reliability	Failure injection	SAG down, LLM down, job retry, duplicate ingestion

Quality Assurance Approach

- **Reviewing:** every change goes through a pull request. Backend review covers architecture, logic, security, validation, error handling, performance and database query; frontend review covers UI/UX, component structure, state, API handling, reuse, responsive behavior and accessibility; database review covers schema, index, migration, backward compatibility and data-loss risk.
- **Tools:** Jest (backend), Vitest and React Testing Library (frontend), Playwright (browser end-to-end), Pytest (AI service).
- **Quality gates:** backend `npm run check` runs lint, typecheck, unit tests, e2e tests and build; frontend `npm run check` runs lint, typecheck, tests and build. GitHub Actions CI checks run on pull requests.
- **Rules:** write a failing test before adding behavior; treat files, webhooks, Jira data, user input and AI output as untrusted; permission filtering happens before evidence is sent to the AI engine.

Quality Metrics (from the evaluation methodology)

Group	Metrics
Retrieval	Recall@5, Recall@10, MRR, Precision@K; supporting-evidence recall for multi-hop questions
Answer	Correctness, faithfulness/groundedness, citation correctness and completeness, answer relevance
Extraction	Knowledge Object precision/recall, entity F1, relationship F1, human acceptance/edit/reject rate
Gap / Conflict	Precision/recall or review acceptance rate for candidate gaps and conflicts
Security	Unauthorized evidence leakage rate (target 0)

Group	Metrics
Performance	Retrieval p50/p95, total response time, ingestion throughput, job failure rate
Cost	Tokens per query, model calls per query
Business proxy	Time-to-information, expert interruption proxy, knowledge coverage change

2.3 Training Plan

No training plan is defined in the source documents. [TBD]

Training Area	Participants	When, Duration	Waiver Criteria
[TBD]	[TBD]	[TBD]	[TBD]
[TBD]	[TBD]	[TBD]	[TBD]
[TBD]	[TBD]	[TBD]	[TBD]

3. Project Deliverables

Due dates use the relative weeks of the 10-week plan; calendar dates are not defined in the source documents.

#	Deliverable	Due	Notes
1	Foundation: scope and actor decisions, DB/API contracts, dataset design, Docker baseline	Week 1	MVP scope and actor/role model are accepted baselines; backend repository has CI checks and the IAM v1 OpenAPI contract; frontend is bootstrapped from TailAdmin (repository history)
2	Access & project: auth, 3 roles, project.create grant, membership, ACL, audit	Week 2–3	
3	Capture & integration: manual note, Jira sync, R2 upload, parse/OCR/job status	Week 3–5	
4	Knowledge lifecycle: SAG index/trace, Proposed Knowledge, human verification, versions	Week 5–7	
5	Chat & handover: permission-aware chat with citations, gaps, handover checklist	Week 7–8	
6	Evaluation & hardening: dataset benchmark, leak tests, failure analysis, reproducible demo and report	Week 9–10	
7	Documents: architecture, data model, API contract, test plan, benchmark dataset	Throughout	Required by the project success criteria
8	Demo scenarios D1–D5 (verified search, multi-hop memory, gap closure, permission, offboarding risk)	Week 10	Defense demo scenarios in the specification

4. Responsibility Assignments

The specification defines five workstreams by role. Assignment of team members and the D/R/S/I matrix are not defined in the source documents.

Workstream	Scope (from the specification)	Assigned member(s)
Product / BA	SRS, use cases, acceptance criteria, benchmark questions, demo narrative	[TBD]
Backend	Domain model, RBAC, workflow, audit, APIs, database	[TBD]
AI / Retrieval	SAG adapter, benchmark, extraction, prompt/schema, evaluation	[TBD]
Frontend	Search/assistant, review center, interview, dashboard, admin	[TBD]
DevOps / QA	Docker, CI, test data, observability, load/security/e2e testing	[TBD]

5. Project Communications

Communication Item	Who / Target	Purpose	When, Frequency	Type, Tool, Method(s)
Task tracking	All team members	Backlog, status and PR links per task	Continuous	Jira
Pull request review	Developers, teammate reviewers	Code review, approval before merge	Per pull request	GitHub pull requests
Database change notice	All team members	Announce collection/field/index/migration changes and data impact	Per database change	Team chat, standard [DATABASE CHANGE] template
Team meetings	[TBD]	[TBD]	[TBD]	[TBD]
Meetings with lecturer	[TBD]	[TBD]	[TBD]	[TBD]

6. Configuration Management

6.1 Document Management

Project documents are versioned as Markdown in the Document repository, the canonical source for the specification, architecture and database design. The backend and frontend repositories keep their task plans and specifications under docs/ and tasks/. The folder structure of the Document repository is:

Folder	Meaning
research-docs	MVP scope, actors/roles/permissions, daily workflow and Jira sync, and the Graduation Project Specification.
research-tech	Approved technology stack and architecture decisions.
architecture	System architecture: topology, frontend, services, storage/messaging/AI, security, orchestration, reliability.
database-design	Schema documents for each service and the schema catalogue.
deploy	Docker Compose files, Kubernetes manifests and backup scripts.
diagram	Interactive architecture diagram.
.agents	Project rules for AI coding agents (workflow and frozen core specifications).
Root files	README, AGENTS.md and AI_WORKFLOW.md (mandatory workflow), and per-tool instruction files.

6.2 Source Code Management

Source Code Management Strategy

- **Repository structure:** separate repositories under the GitHub organization DATN-SPRING2027.
- **Branching:** every task uses a new branch created from the latest origin/main; no direct or force pushes to main; feature branches open pull requests into main only.
- **Branch names:** feat/<Name>-<task>-be-api (backend), feat/<Name>-<task>-fe-ui (frontend), feat/<Name>-<task>-db (database); fix/ or chore/ prefixes for bugs and chores when the team or Jira requires.
- **Separation of concerns:** database, backend and frontend changes use separate branches and pull requests (Figure 2).
- **Commit standards:** atomic commits following Conventional Commits (for example feat:, fix:, docs:, ci:).
- **Code review:** merge only after teammate approval, with no unresolved change requests or conflicts and with passing CI, build and tests.
- **Database changes:** any schema, index, enum, default, reference or validation change is a database change with its own pull request and a new, idempotent migration script; committed migrations are never edited; the team is notified using the standard template.
- **Secrets:** no secrets, tokens or populated .env files are committed; only safe placeholders in .env.example.

Repositories

Repository	Content	Main technology
Document	Specification, architecture, database design, deployment references, workflow rules	Markdown, YAML
DATN-BE	Continuum backend-core modular monolith with eight domain boundaries: iam, capture, jira, lifecycle, chat, handover, ingestion, notification; OpenAPI contracts	NestJS, TypeScript, Jest, Swagger

Repository	Content	Main technology
DATN-FE	Web application built on TailAdmin (dashboard shell, knowledge, assistant, verification, handover, administration screens)	Next.js 16, React 19, Tailwind CSS, TanStack Query, Vitest
AI service	FastAPI service integrating a pinned SAG version behind integrations/ai-engine; repository not present in the workspace: [TBD]	Python, FastAPI, Pytest

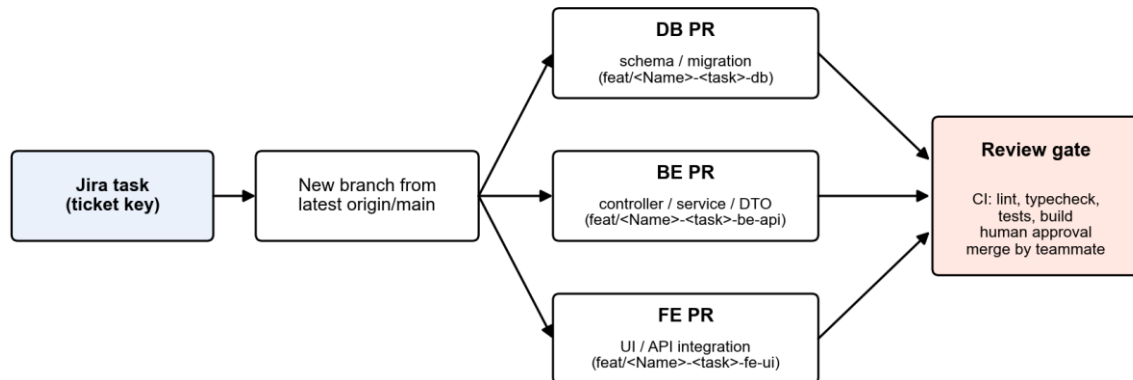


Figure 2. Task delivery flow: Jira task, new branch, separate DB/BE/FE pull requests and review gate.

6.3 Tools & Infrastructures

Category	Tools / Infrastructure
Technology	Next.js (App Router), React, TypeScript (Frontend); Node.js, NestJS, TypeScript (Backend); Python, FastAPI (AI service)
Database and storage	MongoDB with Mongoose (source of truth), LanceDB via SAG (initial retrieval index), Redis, Cloudflare R2 (S3-compatible adapter as fallback)
AI / Retrieval	Pinned zleap-sag, provider-agnostic LLM Gateway, SAG pipeline with MarkItDown or MinerU for parsing and OCR
Background jobs	Redis and BullMQ
Integrations	Jira Cloud REST API, webhooks and reconciliation
UI foundation	TailAdmin (Next.js edition, MIT license) with Tailwind CSS; TanStack Query and Zustand for state
Testing	Jest, Vitest, React Testing Library, Playwright, Pytest
API documentation	Swagger and OpenAPI
Version control	Git and GitHub (source code and documents)
Project management	Jira
CI/CD	GitHub Actions
Deployment	Docker Compose; Kubernetes manifests are kept in Document/deploy/k8s